

AMS 580 Midterm 1

Kachun Huang

2025-03-26

AMS 580 Exam 1

Import Library

```
library(tidyverse)
```

```
## Warning: package 'purrr' was built under R version 4.3.3
```

```
## Warning: package 'lubridate' was built under R version 4.3.3
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
## v dplyr      1.1.4      v readr      2.1.5
```

```
## v forcats   1.0.0      v stringr   1.5.1
```

```
## v ggplot2   3.5.1      v tibble    3.2.1
```

```
## v lubridate 1.9.4      v tidyr     1.3.1
```

```
## v purrr     1.0.4
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.3.3
```

```
## Loading required package: lattice
```

```
## Warning: package 'lattice' was built under R version 4.3.3
```

```
##
```

```
## Attaching package: 'caret'
```

```
##
```

```
## The following object is masked from 'package:purrr':
```

```
##
```

```
## lift
```

```
library(neuralnet)
```

```
## Warning: package 'neuralnet' was built under R version 4.3.3
```

```
##  
## Attaching package: 'neuralnet'  
##  
## The following object is masked from 'package:dplyr':  
##  
##     compute
```

```
library(leaps)
```

```
## Warning: package 'leaps' was built under R version 4.3.3
```

```
library(bestglm)  
library(MASS)
```

```
##  
## Attaching package: 'MASS'  
##  
## The following object is masked from 'package:dplyr':  
##  
##     select
```

```
library(dplyr)
```

Task 1

Import Data

```
unknown <- read.csv("/Users/thomashuang/Downloads/Unknown.csv")  
head(unknown)
```

```
##      u1    u2 u3    u4  u5 u6    u7 u8    u9    u10   u11   u12 y  
## 1 1.59 0.00 0 0.63 0.0 0 1.59 0 0.000 0.275 0.000 0.496 1  
## 2 0.00 0.00 0 0.00 0.0 0 0.00 0 0.000 0.729 0.000 0.000 1  
## 3 0.76 0.15 0 0.00 0.1 0 0.00 0 0.042 0.101 0.016 0.046 1  
## 4 2.94 0.00 0 2.94 0.0 0 0.00 0 0.404 0.404 0.000 0.000 1  
## 5 1.16 0.00 0 1.16 0.0 0 0.00 0 0.000 0.133 0.000 0.000 1  
## 6 0.00 0.00 0 0.00 0.0 0 0.00 0 0.000 0.196 0.000 0.196 1
```

```
str(unknown)
```

```
## 'data.frame':   4621 obs. of  13 variables:  
## $ u1 : num  1.59 0 0.76 2.94 1.16 0 0.76 0 0 0 ...  
## $ u2 : num  0 0 0.15 0 0 0 0.15 0 1.66 0 ...
```

```
## $ u3 : num 0 0 0 0 0 0 0 0 0 0 ...
## $ u4 : num 0.63 0 0 2.94 1.16 0 0 0 0 3.33 ...
## $ u5 : num 0 0 0.1 0 0 0 0.1 0 0 0 ...
## $ u6 : num 0 0 0 0 0 0 0 0 0 0 ...
## $ u7 : num 1.59 0 0 0 0 0 0 0 0 0 ...
## $ u8 : num 0 0 0 0 0 0 0 0 0 0 ...
## $ u9 : num 0 0 0.042 0.404 0 0 0.042 0 0 0 ...
## $ u10: num 0.275 0.729 0.101 0.404 0.133 0.196 0.101 0.196 0 0.352 ...
## $ u11: num 0 0 0.016 0 0 0 0.016 0 0 0 ...
## $ u12: num 0.496 0 0.046 0 0 0.196 0.046 0.196 0 0 ...
## $ y : int 1 1 1 1 1 1 1 1 1 1 ...
```

Data Pre-processing

Remove Missing Values

```
unknown <- na.omit(unknown)
head(unknown)
```

```
##      u1      u2 u3      u4      u5 u6      u7 u8      u9      u10      u11      u12 y
## 1 1.59 0.00 0 0.63 0.0 0 1.59 0 0.000 0.275 0.000 0.496 1
## 2 0.00 0.00 0 0.00 0.0 0 0.00 0 0.000 0.729 0.000 0.000 1
## 3 0.76 0.15 0 0.00 0.1 0 0.00 0 0.042 0.101 0.016 0.046 1
## 4 2.94 0.00 0 2.94 0.0 0 0.00 0 0.404 0.404 0.000 0.000 1
## 5 1.16 0.00 0 1.16 0.0 0 0.00 0 0.000 0.133 0.000 0.000 1
## 6 0.00 0.00 0 0.00 0.0 0 0.00 0 0.000 0.196 0.000 0.196 1
```

```
str(unknown)
```

```
## 'data.frame':    4621 obs. of  13 variables:
## $ u1 : num 1.59 0 0.76 2.94 1.16 0 0.76 0 0 0 ...
## $ u2 : num 0 0 0.15 0 0 0 0.15 0 1.66 0 ...
## $ u3 : num 0 0 0 0 0 0 0 0 0 0 ...
## $ u4 : num 0.63 0 0 2.94 1.16 0 0 0 0 3.33 ...
## $ u5 : num 0 0 0.1 0 0 0 0.1 0 0 0 ...
## $ u6 : num 0 0 0 0 0 0 0 0 0 0 ...
## $ u7 : num 1.59 0 0 0 0 0 0 0 0 0 ...
## $ u8 : num 0 0 0 0 0 0 0 0 0 0 ...
## $ u9 : num 0 0 0.042 0.404 0 0 0.042 0 0 0 ...
## $ u10: num 0.275 0.729 0.101 0.404 0.133 0.196 0.101 0.196 0 0.352 ...
## $ u11: num 0 0 0.016 0 0 0 0.016 0 0 0 ...
## $ u12: num 0.496 0 0.046 0 0 0.196 0.046 0.196 0 0 ...
## $ y : int 1 1 1 1 1 1 1 1 1 1 ...
```

After cleaning missing values, there are 4621 observations left.

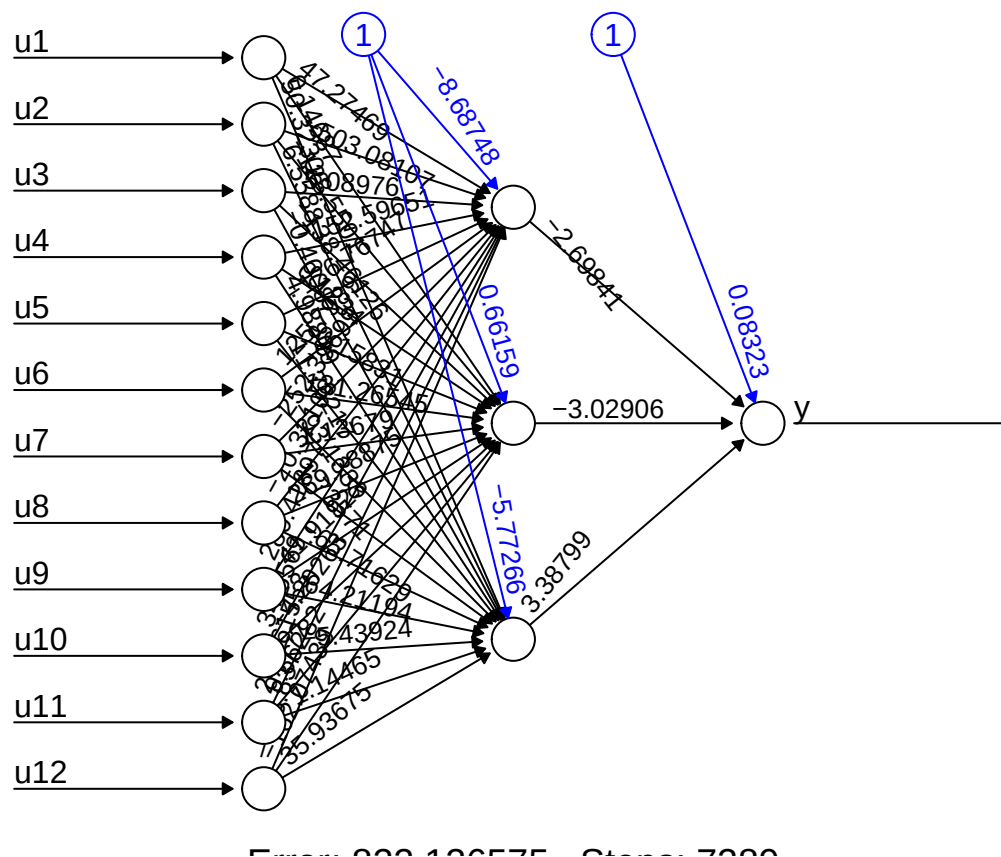
Data Splitting

```
set.seed(123)
training.sample <- unknown$y %>% createDataPartition(p=0.75, list=FALSE)
train.data <- unknown[training.sample,]
test.data <- unknown[-training.sample,]
```

NN Model with 1 hidden

```
nn_one_model_task1 <- neuralnet(
  y ~.,
  train.data,
  hidden = 3,
  err.fct = "ce",
  linear.output = FALSE
)
```

```
plot(nn_one_model_task1, rep = "best")
```



Predict on Test Data

```
pred_nn_one_task1 <- predict(nn_one_model_task1, test.data)
pred_nn_one_task1_class <- ifelse(pred_nn_one_task1 > 0.5, 1, 0)
```

Confusion Matrix

```
cm1 <- confusionMatrix(factor(pred_nn_one_task1_class), factor(test.data$y), positive = "1")
cm1
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 641  66
##           1  47 401
##
##           Accuracy : 0.9022
##           95% CI : (0.8836, 0.9187)
##       No Information Rate : 0.5957
##       P-Value [Acc > NIR] : <2e-16
##
##           Kappa : 0.7956
##
##  Mcnemar's Test P-Value : 0.0904
##
##           Sensitivity : 0.8587
##           Specificity : 0.9317
##           Pos Pred Value : 0.8951
##           Neg Pred Value : 0.9066
##           Prevalence : 0.4043
##           Detection Rate : 0.3472
##       Detection Prevalence : 0.3879
##           Balanced Accuracy : 0.8952
##
##           'Positive' Class : 1
##
```

sensitivity

```
cm1$byClass["Sensitivity"]
```

```
## Sensitivity
##    0.8586724
```

specificity

```
cm1$byClass["Specificity"]
```

```
## Specificity
##    0.931686
```

overall accuracy

```
cm1$overall["Accuracy"]
```

```
## Accuracy  
## 0.9021645
```

Logistic Model

```
log_model <- glm(y ~., data=train.data, family = binomial)
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
summary(log_model)
```

```
##  
## Call:  
## glm(formula = y ~ ., family = binomial, data = train.data)  
##  
## Coefficients:  
##              Estimate Std. Error z value Pr(>|z|)  
## (Intercept) -1.19301    0.07300 -16.343 < 2e-16 ***  
## u1           0.41781    0.07553   5.531 3.18e-08 ***  
## u2           4.68280    0.47767   9.803 < 2e-16 ***  
## u3           0.29264    0.07602   3.850 0.000118 ***  
## u4           0.71018    0.09351   7.594 3.09e-14 ***  
## u5           2.68618    0.53379   5.032 4.85e-07 ***  
## u6          -2.95898    0.31774  -9.313 < 2e-16 ***  
## u7           0.30453    0.31118   0.979 0.327765  
## u8          -1.89841    0.32846  -5.780 7.48e-09 ***  
## u9          -0.42296    0.28673  -1.475 0.140170  
## u10          -0.24237    0.22895  -1.059 0.289794  
## u11          -1.58580    1.03784  -1.528 0.126517  
## u12          10.73737    0.74267  14.458 < 2e-16 ***  
## ---  
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1  
##  
## (Dispersion parameter for binomial family taken to be 1)  
##  
##    Null deviance: 4639.6  on 3465  degrees of freedom  
## Residual deviance: 2334.8  on 3453  degrees of freedom  
## AIC: 2360.8  
##  
## Number of Fisher Scoring iterations: 8
```

Predict on Test Data

```
pred_prob <- log_model %>% predict(test.data, type="response")
predicted.classes <- ifelse(pred_prob > 0.5, "1", "0")
```

Confusion Matrix

```
cm2 <- confusionMatrix(factor(predicted.classes), factor(test.data$y), positive = "1")
cm2
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction    0    1
##              0 655 123
##              1  33 344
##
##              Accuracy : 0.8649
##              95% CI : (0.8439, 0.8841)
##              No Information Rate : 0.5957
##              P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.7106
##
##              Mcnemar's Test P-Value : 1.036e-12
##
##              Sensitivity : 0.7366
##              Specificity : 0.9520
##              Pos Pred Value : 0.9125
##              Neg Pred Value : 0.8419
##              Prevalence : 0.4043
##              Detection Rate : 0.2978
##              Detection Prevalence : 0.3264
##              Balanced Accuracy : 0.8443
##
##              'Positive' Class : 1
##
```

sensitivity

```
cm2$byClass["Sensitivity"]
```

```
## Sensitivity
## 0.7366167
```

specificity

```
cm2$byClass["Specificity"]
```

```
## Specificity  
## 0.9520349
```

overall accuracy

```
cm2$overall["Accuracy"]
```

```
## Accuracy  
## 0.8649351
```

Write Dataset File

```
predictions <- data.frame(  
  true_obs = test.data$y,  
  nn_pred = pred_nn_one_task1_class,  
  log_pred = predicted.classes  
)  
agreement_table <- table(predictions$nn_pred, predictions$log_pred)  
agreement_table
```

```
##  
##      0      1  
## 0 695  12  
## 1  83 365
```

Result

The result said there is a very high agreement rate and low disagreement.

Task 2

Import Data

```
madness <- read.csv("/Users/thomashuang/Downloads/Madness.csv")  
head(madness)
```

```
##   x1    x2 x3 x4   x5   x6   x7 x8 x9 x10 x11 x12 x13 x14 x15    y  
## 1  5  84.50  7  5  856  854 1710  2  3   1   8   0   2  548   y 2140  
## 2 31  96.00  6  8 1262    0 1262  2  3   1   6   1   2  460   y 1870  
## 3  7 112.50  7  5  920  866 1786  2  3   1   6   1   2  608   y 2290  
## 4 91  95.50  7  5  961  756 1717  1  3   1   7   1   3  642   y 1455  
## 5  8 142.60  8  5 1145 1053 2198  2  4   1   9   1   3  836   y 2555  
## 6 16 141.15  5  5  796  566 1362  1  1   1   5   0   2  480   y 1485
```

```
str(madness)
```

```
## 'data.frame': 1460 obs. of 16 variables:
## $ x1 : int 5 31 7 91 8 16 3 36 77 69 ...
## $ x2 : num 84.5 96 112.5 95.5 142.6 ...
## $ x3 : int 7 6 7 7 8 5 8 7 7 5 ...
## $ x4 : int 5 8 5 5 5 5 5 6 5 6 ...
## $ x5 : int 856 1262 920 961 1145 796 1694 1107 1022 1077 ...
## $ x6 : int 854 0 866 756 1053 566 0 983 752 0 ...
## $ x7 : int 1710 1262 1786 1717 2198 1362 1694 2090 1774 1077 ...
## $ x8 : int 2 2 2 1 2 1 2 2 2 1 ...
## $ x9 : int 3 3 3 3 4 1 3 3 2 2 ...
## $ x10: int 1 1 1 1 1 1 1 1 2 2 ...
## $ x11: int 8 6 6 7 9 5 7 7 8 5 ...
## $ x12: int 0 1 1 1 1 0 1 2 2 2 ...
## $ x13: int 2 2 2 3 3 2 2 2 2 1 ...
## $ x14: int 548 460 608 642 836 480 636 484 468 205 ...
## $ x15: chr "y" "y" "y" "y" ...
## $ y : num 2140 1870 2290 1455 2555 ...
```

Data Pre-processing

Remove Missing Values

```
madness <- na.omit(madness)
```

```
str(madness)
```

```
## 'data.frame': 1460 obs. of 16 variables:
## $ x1 : int 5 31 7 91 8 16 3 36 77 69 ...
## $ x2 : num 84.5 96 112.5 95.5 142.6 ...
## $ x3 : int 7 6 7 7 8 5 8 7 7 5 ...
## $ x4 : int 5 8 5 5 5 5 5 6 5 6 ...
## $ x5 : int 856 1262 920 961 1145 796 1694 1107 1022 1077 ...
## $ x6 : int 854 0 866 756 1053 566 0 983 752 0 ...
## $ x7 : int 1710 1262 1786 1717 2198 1362 1694 2090 1774 1077 ...
## $ x8 : int 2 2 2 1 2 1 2 2 2 1 ...
## $ x9 : int 3 3 3 3 4 1 3 3 2 2 ...
## $ x10: int 1 1 1 1 1 1 1 1 2 2 ...
## $ x11: int 8 6 6 7 9 5 7 7 8 5 ...
## $ x12: int 0 1 1 1 1 0 1 2 2 2 ...
## $ x13: int 2 2 2 3 3 2 2 2 2 1 ...
## $ x14: int 548 460 608 642 836 480 636 484 468 205 ...
## $ x15: chr "y" "y" "y" "y" ...
## $ y : num 2140 1870 2290 1455 2555 ...
```

After cleaning, there are 1460 observations left.

Mean and std

```
mean <- mean(madness$y)
print(mean)
```

```
## [1] 1864.212
```

```
sd <- sd(madness$y)
print(sd)
```

```
## [1] 794.425
```

factorize variable

```
categ_col <- ifelse(madness$x15 == "y", 1, 0)
```

Scale the Data

```
madness <- data.frame(scale(subset(madness, select = -x15)))

madness$x15 <- categ_col

head(madness)
```

```
##           x1           x2           x3           x4           x5           x6
## 1 -1.0429020 -0.20707076  0.65125610 -0.5170227 -0.79316202  1.1614536
## 2 -0.1834022 -0.09185490 -0.07181151  2.1788812  0.25705235 -0.7948909
## 3 -0.9767867  0.07345481  0.65125610 -0.5170227 -0.62761099  1.1889432
## 4  1.8000589 -0.09686428  0.65125610 -0.5170227 -0.52155486  0.9369551
## 5 -0.9437290  0.37501979  1.37432370 -0.5170227 -0.04559563  1.6173231
## 6 -0.6792675  0.36049258 -0.79487911 -0.5170227 -0.94836612  0.5017028
##           x7           x8           x9           x10          x11          x12          x13
## 1  0.3702066  0.789470  0.163723 -0.2113812  0.9118973 -0.9509007  0.3116179
## 2 -0.4823466  0.789470  0.163723 -0.2113812 -0.3185741  0.6002892  0.3116179
## 3  0.5148362  0.789470  0.163723 -0.2113812 -0.3185741  0.6002892  0.3116179
## 4  0.3835277 -1.025689  0.163723 -0.2113812  0.2966616  0.6002892  1.6497417
## 5  1.2988806  0.789470  1.389547 -0.2113812  1.5271330  0.6002892  1.6497417
## 6 -0.2920446 -1.025689 -2.287924 -0.2113812 -0.9338098 -0.9509007  0.3116179
##           x14           y x15
## 1  0.35088009  0.347154270  1
## 2 -0.06071021  0.007285824  1
## 3  0.63150985  0.535970074  1
## 4  0.79053338 -0.515104565  1
## 5  1.69790291  0.869544659  1
## 6  0.03283304 -0.477341404  1
```

Data Splitting

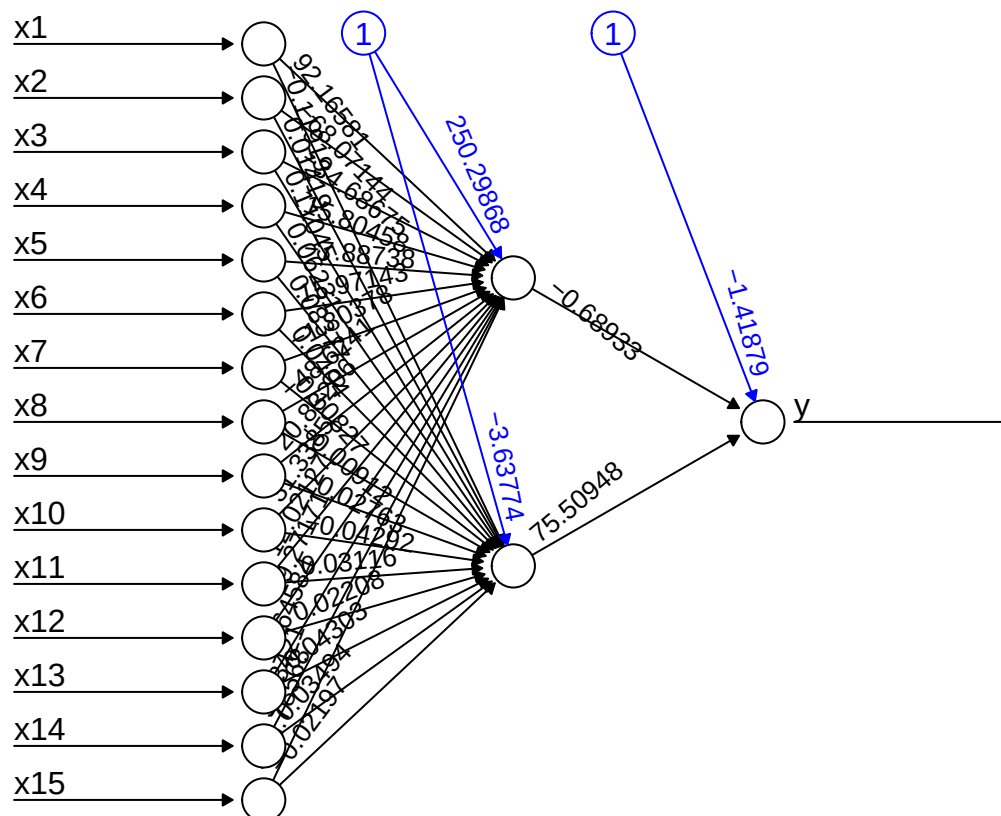
```
set.seed(123)
training.sample <- madness$y %>% createDataPartition(p=0.75, list=FALSE)
train.data <- madness[training.sample,]
test.data <- madness[-training.sample,]
```

NN Model with 1 hidden

```
nn_one_model_task2 <- neuralnet(  
  y ~.,  
  train.data,  
  hidden = 2,  
  err.fct = "sse",  
  linear.output = TRUE,  
  stepmax = 1e6  
)
```

plot

```
plot(nn_one_model_task2, rep="best")
```



Predict on Test Data

```
pred_one_model_task2 <- predict(nn_one_model_task2, test.data)
```

Descale

```
# compute y(unscaled)  
y <- sd * test.data$y + mean  
  
#compute y^hat (unscaled)  
yhat <- pred_one_model_task2*sd+mean
```

Compute Test MSE

```
sum((y-yhat)^2)/length(y)
```

```
## [1] 740794.1
```

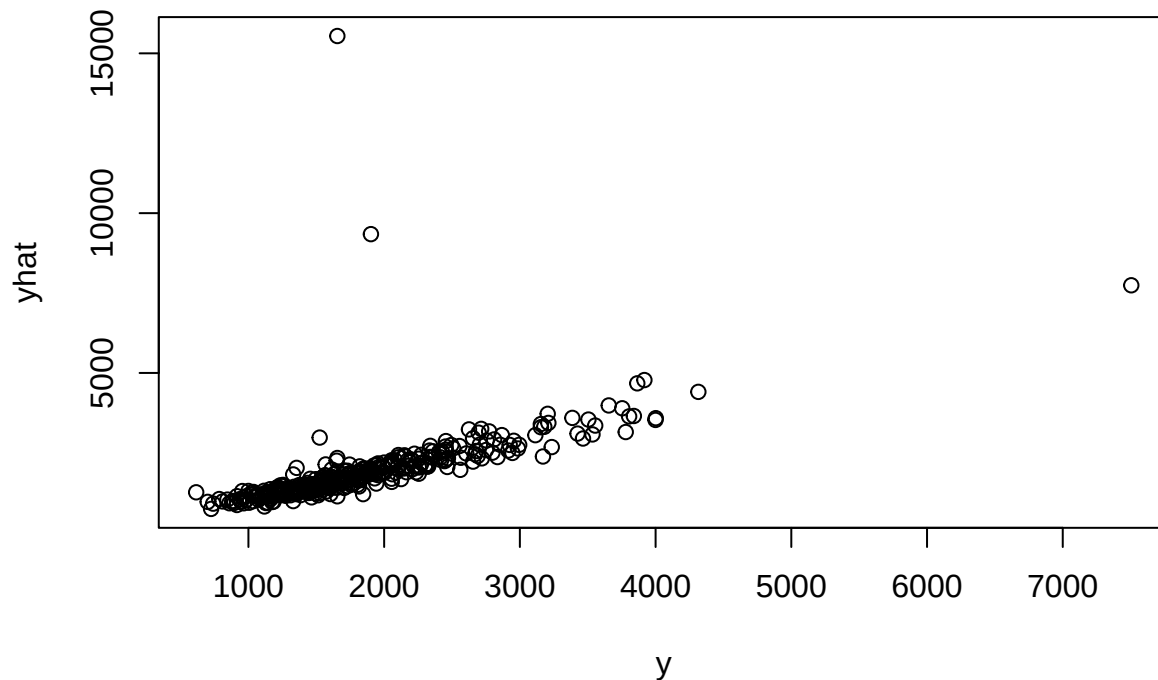
or

```
(MSE.nn.1 = RMSE(y,yhat)^2)
```

```
## [1] 740794.1
```

Plot $\hat{y}$ and y

```
plot(y, yhat)
```



Linear Regression

```
# null model
null <- lm(y ~ 1, data=train.data)

#full model
full <-lm(y ~. , data=train.data)
```

Bidirectional

```
#find the best model in backward stepwise
fit_step_both_bic <- stepAIC(full, k = log(nrow(train.data)), trace = 1,direction="both")
```

```
## Start:  AIC=-1831.91
## y ~ x1 + x2 + x3 + x4 + x5 + x6 + x7 + x8 + x9 + x10 + x11 +
##      x12 + x13 + x14 + x15
##
##      Df Sum of Sq  RSS   AIC
## - x15   1    0.000 186.48 -1838.9
## - x13   1    0.032 186.51 -1838.7
## - x12   1    0.195 186.67 -1837.8
## - x7    1    0.299 186.78 -1837.2
## - x6    1    0.518 187.00 -1835.9
## - x8    1    0.835 187.31 -1834.0
## <none>          186.48 -1831.9
## - x11   1    1.671 188.15 -1829.1
## - x5    1    1.909 188.39 -1827.7
```

```

## - x14 1 2.469 188.95 -1824.5
## - x10 1 5.654 192.13 -1806.1
## - x2 1 5.909 192.39 -1804.7
## - x4 1 6.397 192.87 -1801.9
## - x9 1 11.207 197.68 -1774.9
## - x1 1 19.182 205.66 -1731.5
## - x3 1 32.906 219.38 -1660.6
##
## Step: AIC=-1838.91
## y ~ x1 + x2 + x3 + x4 + x5 + x6 + x7 + x8 + x9 + x10 + x11 +
## x12 + x13 + x14
##
## Df Sum of Sq RSS AIC
## - x13 1 0.032 186.51 -1845.7
## - x12 1 0.195 186.67 -1844.8
## - x7 1 0.299 186.78 -1844.2
## - x6 1 0.518 187.00 -1842.9
## - x8 1 0.841 187.32 -1841.0
## <none> 186.48 -1838.9
## - x11 1 1.671 188.15 -1836.1
## - x5 1 1.909 188.39 -1834.7
## + x15 1 0.000 186.48 -1831.9
## - x14 1 2.469 188.95 -1831.5
## - x10 1 5.660 192.14 -1813.1
## - x2 1 5.910 192.39 -1811.7
## - x4 1 6.408 192.88 -1808.8
## - x9 1 11.233 197.71 -1781.7
## - x1 1 19.182 205.66 -1738.5
## - x3 1 32.963 219.44 -1667.3
##
## Step: AIC=-1845.72
## y ~ x1 + x2 + x3 + x4 + x5 + x6 + x7 + x8 + x9 + x10 + x11 +
## x12 + x14
##
## Df Sum of Sq RSS AIC
## - x12 1 0.184 186.69 -1851.6
## - x7 1 0.305 186.81 -1850.9
## - x6 1 0.511 187.02 -1849.7
## - x8 1 0.869 187.38 -1847.6
## <none> 186.51 -1845.7
## - x11 1 1.654 188.16 -1843.0
## - x5 1 1.899 188.41 -1841.6
## + x13 1 0.032 186.48 -1838.9
## + x15 1 0.000 186.51 -1838.7
## - x14 1 5.593 192.10 -1820.3
## - x10 1 5.738 192.25 -1819.5
## - x2 1 5.895 192.40 -1818.6
## - x4 1 6.435 192.94 -1815.5
## - x9 1 11.201 197.71 -1788.7
## - x1 1 19.355 205.86 -1744.4
## - x3 1 33.116 219.62 -1673.4
##
## Step: AIC=-1851.64
## y ~ x1 + x2 + x3 + x4 + x5 + x6 + x7 + x8 + x9 + x10 + x11 +

```

```

##      x14
##
##      Df Sum of Sq    RSS      AIC
## - x7      1      0.287 186.98 -1857.0
## - x6      1      0.565 187.26 -1855.3
## - x8      1      0.922 187.61 -1853.2
## <none>                186.69 -1851.6
## - x11     1      1.655 188.35 -1849.0
## - x5      1      2.044 188.74 -1846.7
## + x12     1      0.184 186.51 -1845.7
## + x13     1      0.020 186.67 -1844.8
## + x15     1      0.000 186.69 -1844.6
## - x14     1      5.524 192.22 -1826.7
## - x10     1      6.073 192.76 -1823.5
## - x2      1      6.353 193.05 -1821.9
## - x4      1      6.423 193.12 -1821.5
## - x9      1     11.515 198.21 -1793.0
## - x1      1     19.219 205.91 -1751.2
## - x3      1     34.139 220.83 -1674.4
##
## Step: AIC=-1856.96
## y ~ x1 + x2 + x3 + x4 + x5 + x6 + x8 + x9 + x10 + x11 + x14
##
##      Df Sum of Sq    RSS      AIC
## - x8      1      0.894 187.87 -1858.7
## <none>                186.98 -1857.0
## - x11     1      1.908 188.89 -1852.8
## + x7      1      0.287 186.69 -1851.6
## + x12     1      0.166 186.81 -1850.9
## + x13     1      0.025 186.95 -1850.1
## + x15     1      0.000 186.98 -1850.0
## - x14     1      5.607 192.59 -1831.5
## - x4      1      6.282 193.26 -1827.7
## - x2      1      6.328 193.31 -1827.5
## - x10     1      6.404 193.38 -1827.0
## - x9      1     11.655 198.63 -1797.6
## - x1      1     19.047 206.03 -1757.5
## - x3      1     34.500 221.48 -1678.2
## - x6      1     39.019 226.00 -1656.0
## - x5      1     74.486 261.46 -1496.1
##
## Step: AIC=-1858.73
## y ~ x1 + x2 + x3 + x4 + x5 + x6 + x9 + x10 + x11 + x14
##
##      Df Sum of Sq    RSS      AIC
## <none>                187.87 -1858.7
## + x8      1      0.894 186.98 -1857.0
## - x11     1      1.944 189.82 -1854.4
## + x7      1      0.259 187.61 -1853.2
## + x12     1      0.217 187.66 -1853.0
## + x13     1      0.049 187.82 -1852.0
## + x15     1      0.007 187.87 -1851.8
## - x14     1      5.580 193.45 -1833.6
## - x4      1      6.316 194.19 -1829.5

```

```
## - x2      1      6.358 194.23 -1829.2
## - x10     1      7.775 195.65 -1821.2
## - x9      1     12.733 200.61 -1793.8
## - x1      1     18.484 206.36 -1762.8
## - x3      1     33.659 221.53 -1684.9
## - x6      1     39.073 226.95 -1658.5
## - x5      1     74.900 262.77 -1497.7
```

```
fit_step_both_bic
```

```
##
## Call:
## lm(formula = y ~ x1 + x2 + x3 + x4 + x5 + x6 + x9 + x10 + x11 +
##      x14, data = train.data)
##
## Coefficients:
## (Intercept)          x1          x2          x3          x4          x5
##    0.01071    -0.18337     0.09195     0.29024     0.08378     0.49322
##          x6          x9          x10          x11          x14
##    0.35574    -0.15297    -0.09244     0.08930     0.09560
```

Predict on Test Data

```
pred_stepwise <- predict(fit_step_both_bic, test.data)
```

Descalce

```
# compute y(unscaled)
y <- sd * test.data$y + mean

#compute y^hat (unscaled)
yhat <- pred_stepwise*sd+mean
```

Compute Test MSE

```
sum((y-yhat)^2)/length(y)
```

```
## [1] 237563.5
```

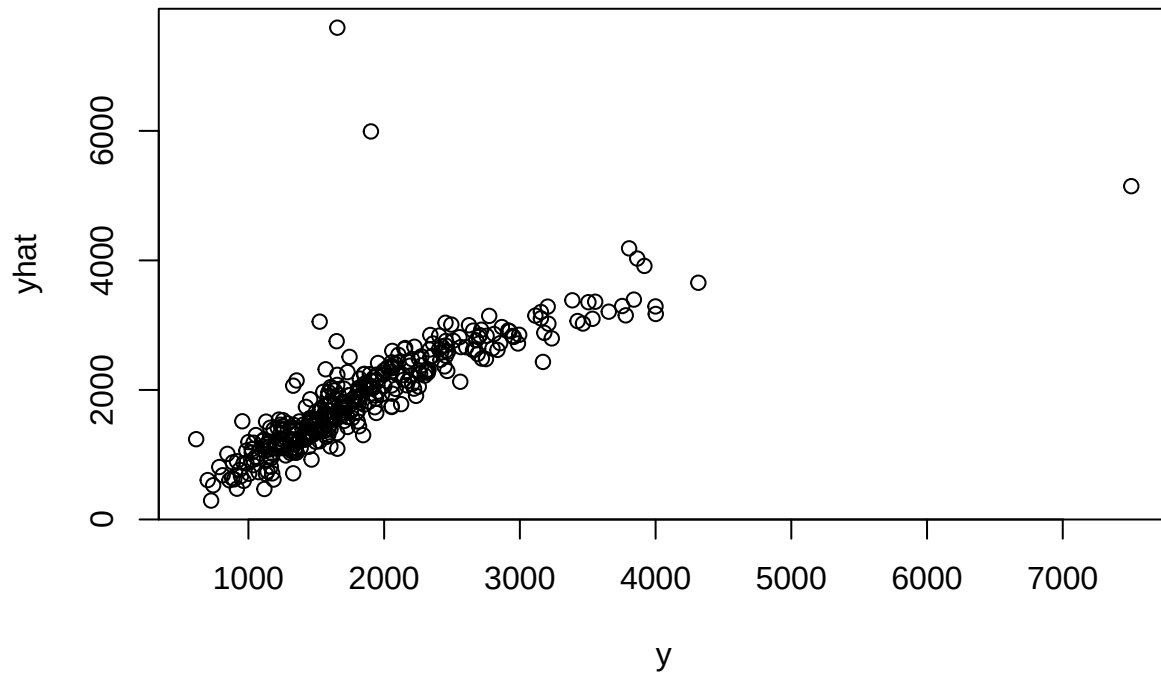
or

```
(MSE.stepwise = RMSE(y,yhat)^2)
```

```
## [1] 237563.5
```

Plot $\hat{y}$ and y

```
plot(y, yhat)
```



Task 2 Result

In the Madness dataset, the stepwise linear regression performs better than neural network with 1 hidden layer and 2 neurons. The possible reason behind this could be

- Overfitting: with 1 hidden layer and 2 neurons could overfit on the training data and perform worse in testing data
- Linear Relationship: it is possible that the dataset already explain well in linear model, so neural network might introduces some unnecessary relationship to the model